

MODULE 3 · LEVEL 300



Design: Architecture & UX with AI

From an agreed spec to a buildable design - architecture, diagrams, data & API contracts, and prototypes, with GitHub Copilot

Day 1 · 90 minutes · Builds on Module 2 (the spec) · Novnex



Where we are, and today's shape

⌚ approx 2 min

Module 2 produced an agreed spec. Today we turn it into a design that's ready to build - then Module 4 builds it.



Framing

What design means here, where AI helps, and the golden rule.

~16 min



Design with Copilot

Architecture, diagrams, data & API, prototypes, ADRs, security.

~51 min



Apply

By track, anti-patterns, review, the hand-off, and the lab.

~23 min

Same guardrails as always: ground the context, review everything, and the architect owns the decisions.



Learning objectives

 approx 2 min

By the end of this module you will be able to:



Explore architecture with AI

Use Copilot to propose and compare architecture options and trade-offs from a spec.



Design as code

Produce diagrams, data models and API contracts you can version, review and evolve.



Prototype the experience

Turn a spec or a mockup into an interactive prototype to validate UX early.



Decide and record

Weigh risk and security, choose deliberately, and capture decisions as ADRs - human-owned.



Why design is where AI leverage compounds

⌚ approx 4 min

The design squeeze

- **Blank page:**
 - turning a spec into an architecture is slow and intimidating.
- **Too few options:**
 - teams anchor on the first idea and miss better ones.
- **Undocumented decisions:**
 - the 'why' is lost, so it's re-litigated later.
- **Design drifts from code:**
 - diagrams go stale the day after the workshop.

Why AI helps here

- Generating and comparing options is fast and cheap now
- Design-as-code keeps diagrams and contracts next to the code
- A design decision is far cheaper to change than built code
- A clear design is the target agents and developers build against



Where AI helps across design

 approx 4 min



Architecture

Propose & compare options; trade-off analysis.



Diagrams as code

C4, sequence, flow - from spec or from code.



Data modelling

Schemas & ER models from requirements.



API / contracts

Contract-first OpenAPI from the stories.



UX prototyping

Spark: spec or mockup to an interactive app.



Risk & security

Threat modelling, risky-area and NFR checks.

We'll take these in order. Throughout: Copilot drafts options, the architect decides and records why.



The golden rule: AI proposes, the architect decides

⌚ approx 3 min

Why this rule matters in design

- **Context AI lacks:**
 - your team, budget, existing systems, org constraints.
- **Reversibility:**
 - weigh which decisions are one-way doors vs easy to change.
- **Confidence gap:**
 - a fluent architecture can still be wrong for you.
- **Accountability:**
 - someone owns the design; that someone is human.

Capture it: an ADR

- Architecture Decision Record: a short, versioned note
- Context - the forces and constraints
- Decision - what you chose
- Consequences - the trade-offs you accepted
- Copilot drafts it; you confirm and commit it

PART 2

Design with Copilot

Architecture, diagrams, data and API contracts, prototypes, decisions and security - with real examples.

approx 51 min





From spec to architecture options

⌚ approx 5 min

Give Copilot the spec and ask for two or three architectures with explicit trade-offs - then you choose.

Prompt to Copilot Chat

```
From this spec, propose 3 architecture
options for the self-service portal.
For each: a diagram sketch, and trade-offs
on complexity, cost, scalability, team fit.
Recommend one and say why. <paste spec>
-> A. Modular monolith B. Serverless
    C. Microservices (+ trade-off table)
```

Use it well

- **Force options:**
 - asking for three beats anchoring on one.
- **Name the axes:**
 - complexity, cost, scale, team fit, reversibility.
- **Push back:**
 - ask 'what breaks at 10x load?' or 'what's the risk?'
- **You choose:**
 - pick against your real constraints, then write an ADR.



Diagrams as code, both directions

🕒 approx 5 min

Copilot writes Mermaid (and PlantUML) from your spec - or reverse-engineers a diagram from existing code. It lives in the repo.

Mermaid - generated sequence diagram

```
sequenceDiagram
    Customer->>Portal: Request reset
    Portal->>Auth: Create single-use token
    Auth-->>Portal: token + 30-min expiry
    Portal->>Email: Send reset link
    Email-->>Customer: Reset link
    Customer->>Portal: Set new password
```

Why design-as-code wins

- **Versioned:**
 - diagrams live in Git, reviewed in PRs.
- **Two-way:**
 - forward from spec, reverse from code.
- **Renders everywhere:**
 - GitHub and VS Code show it.
- **Stays current:**
 - update the text, not a drawing tool.



Design patterns - and anti-patterns

⌚ approx 4 min



Suggest & apply

- **Fit a pattern:**
- 'what pattern suits ret/retry with backoff here?'
- **Apply it:**
- scaffold the structure consistently across the code.
- **Explain trade-offs:**
- when the pattern helps and when it's overkill.
- **Learn:**
- a fast way to level up the team on unfamiliar patterns.



Detect & avoid

- **Spot anti-patterns:**
- god objects, tight coupling, leaky abstractions.
- **Flag over-engineering:**
- a pattern where a function would do.
- **Review a design:**
- 'critique this for maintainability and coupling.'
- **Judgement stays yours:**
- patterns are tools, not goals.



Data modelling from requirements

⌚ approx 4 min

Turn the glossary and stories into a data model - Copilot drafts the ER model; you validate keys, grain and PII.

Mermaid - generated ER model

```
erDiagram
    CUSTOMER ||--o{ TICKET : raises
    CUSTOMER {
        uuid id PK
        string email
        datetime last_login
    }
    TICKET { uuid id PK string status }
```

Then you check

- **Grain & keys:**
 - is the primary key and grain right?
- **Definitions:**
 - match the glossary (active user = 30 days).
- **PII & privacy:**
 - flag personal data; apply retention.
- **Normalisation:**
 - fit for the workload, not dogma.



Contract-first API design

⌚ approx 4 min

Design the interface before the implementation. Copilot drafts an OpenAPI contract from the story; teams build against it in parallel.

OpenAPI (draft) - password reset

```
POST /password-reset
  requestBody: { email: string }
  responses:
    202: reset link sent
    404: unknown email (generic message)
    429: too many requests (rate limited)
```

Why contract-first

- **Parallel work:**
 - front-end and back-end agree, then build.
- **Testable:**
 - the contract drives mocks and tests.
- **Consistent:**
 - same error and auth conventions across APIs.
- **Review the contract:**
 - argue here, before code exists.



Prototype the experience with GitHub Spark

⌚ approx 5 min

Prompt (or upload a mockup)

```
Build a self-service account page:  
reset password, change email, close  
account. Match our brand; mobile-first.
```

You get an interactive prototype

- A working UI to click through and react to
- Iterate by prompt, visual edits, or code
- From a screenshot/sketch too; sync to a repo

Design value - and the line

- **Validate UX early:**
 - react to something real, not a static wireframe.
- **Cheap divergence:**
 - try three layouts in minutes.
- **Shared understanding:**
 - stakeholders see it, not imagine it.
- **A prototype, not the product:**
 - treat output as a sketch; preview status; uses credits.



Design systems & component structure

⌚ approx 3 min



Scaffold to the system

Generate components that follow your design system and naming - not ad hoc.



Structure & composition

Propose a sensible component tree and where state lives.



Reuse over reinvention

Grounded in a Space, it reuses existing components instead of new ones.



Consistency by default

Same tokens, spacing and patterns across screens.



Record the decision: ADRs with Copilot

⌚ approx 3 min

ADR (Copilot draft, you confirm)

```
# ADR 007: Single-use, 30-min reset tokens
Status: Accepted
Context: self-service reset; security
        requires expiry and single use.
Decision: single-use token, 30-min TTL,
        stored hashed; generic 404 on unknown.
Consequences: + secure - user must act fast
```

Why ADRs matter

- **The 'why' survives:**
 - future you, and agents, get the reasoning.
- **Fast to write:**
 - Copilot drafts from the discussion.
- **Versioned:**
 - lives in the repo next to the code.
- **Feeds the Space:**
 - past ADRs ground future design answers.



Ground design in a Space

⌚ approx 4 min

Point Copilot at your real architecture so its options fit your world - your standards, systems and past decisions.

Space: "Architecture & Standards"

```
repos:  acme/platform, acme/design-system
files:  architecture.md, tech-radar.md,
        nfr-standards.md, past-ADRs/
instructions:
  Prefer our approved stack (tech radar).
  Reuse existing services & components.
  Apply the NFR checklist; cite ADRs.
```

What changes

- **Fits your stack:**
 - suggestions respect your tech radar, not trends.
- **Reuses systems:**
 - integrates with what exists; less duplication.
- **Consistent standards:**
 - NFRs and conventions applied every time.
- **Decision-aware:**
 - honours past ADRs instead of contradicting them.



Risk and security by design

⌚ approx 5 min



Threat modelling

Ask Copilot to walk STRIDE over the design: spoofing, tampering, info disclosure, DoS.



Risky-area prediction

Flag the complex, high-change or high-blast-radius parts to design more carefully.



NFRs as first-class

Check the design against performance, privacy, accessibility and reliability up front.



Secure-by-design choices

Bake in least privilege, input validation, secrets handling - not bolted on later.



Live demo: spec to a buildable design

⌚ approx 5 min

Watch the design take shape

- Spec -> three architecture options + trade-offs
- Chosen option -> a Mermaid diagram in the repo
- Requirements -> a draft ER model and API contract
- A STRIDE pass surfaces a threat we design against
- We record the decision as an ADR - and edit it

Flow

1.
 - Ask for 3 architectures.
2.
 - Generate the diagram.
3.
 - Draft ER + API contract.
4.
 - Run a STRIDE pass.
5.
 - Write & fix an ADR.



Responsible design with AI

⌚ approx 3 min



Plausible but wrong

A clean architecture that doesn't fit your context.

Guard: Judge against your team, systems and constraints; ground in a Space.



Over-engineering

Patterns and services where simpler would win.

Guard: Ask 'what's the simplest thing that works?'; prefer reversible choices.



Undocumented decisions

Choices made in chat, never recorded.

Guard: Write an ADR for every significant decision; commit it.



Security as an afterthought

Threats and NFRs deferred to later.

Guard: Threat-model and check NFRs at design time, not deploy time.

PART 3

Apply it

By track, the traps to avoid, reviewing a design, the hand-off to development, and your lab.

approx 23 min





Design with Copilot - by track

 approx 4 min



Web / Drupal

Site & module architecture, content model, template structure, integration points.



Data & Analytics

Data model & pipeline architecture; semantic layer; metric definitions to schema.



Power BI

Star-schema / semantic-model design, measure design, row-level-security design.



RPA

Solution design: as-is to to-be process, exception paths, and integration points.



Anti-patterns to avoid

 approx 3 min



Don't

- Accept the first architecture Copilot offers
- Let it choose your stack for you
- Add services/patterns because generating is cheap
- Leave decisions in chat, undocumented
- Skip threat modelling and NFRs until later



Do

- Compare 2-3 options against real constraints
- Ground design in a Space; respect the tech radar
- Prefer the simplest, most reversible design
- Record every significant decision as an ADR
- Design for risk and NFRs up front



Review the design - with AI as a second pair of eyes

 approx 3 min

Before you commit, ask Copilot to critique the design against this checklist - then a human signs off.



Meets the spec

Every requirement and acceptance criterion is addressed.



Fits standards

Consistent with the tech radar, patterns and past ADRs.



Risk & security

Threats considered; NFRs (perf, privacy, a11y) covered.



Clear & current

Diagrams match intent; contracts and data model are explicit.



The hand-off to development

 approx 4 min

A good design package is what Module 4 - and the Copilot agents - build against. Copilot can assemble and summarise it.



Architecture + rationale

The chosen option and why (ADRs).



Diagrams

C4 / sequence / ER, versioned in the repo.



Contracts & data model

OpenAPI and the schema to build to.



Risks & NFRs

Threats to mitigate; non-functional targets.

Next module turns this into working software - with agent mode and the cloud agent building against your design.



Your lab: design a slice, end to end

 approx 2 min

Lab 3 (this afternoon) - on your own track

- Start from the spec you built in Lab 2 (or a provided one) for your track
- Ask Copilot for 2-3 architecture options; choose one against your constraints
- Generate a diagram and a draft data model or API contract; ground it in the Space
- Run a quick risk pass and record one decision as an ADR

Role groups; pair non-coders with developers. The goal is a small, documented, build-ready design.



Discussion & reflection

 approx 4 min

Talk to the person next to you - three minutes:

- Which design task would you most trust Copilot to accelerate - and which would you keep firmly human?
- How would you make sure design decisions get recorded, not lost in a chat window?

Options vs decisions

Over-engineering

Standards & reuse

Documenting

Security

We'll take a few answers before wrapping up.



Key takeaways

- Turn the spec into a design: architecture options, diagrams, data and API contracts.
- Design as code - versioned, reviewable, and it stays current.
- Prototype UX early with Spark; ground everything in a Space.
- Design for risk and NFRs up front; record decisions as ADRs.
- AI proposes options; the architect decides, and owns it.

What's next

Module 4

Development

then

Lab 3 this afternoon

Questions?